

C PROGRAMMING

Practical Index — Coding Questions, Programs & Notes

31 Questions · Arrays, Searching, Sorting, Matrices, Stacks, Queues & Linked Lists

How to use this document

Every question has its own heading (Q1, Q2, ...) so it appears as its own line in the Table of Contents below, with a page number.

Right-click the Table of Contents and choose "Update Field" if page numbers look off after editing.

Table of Contents

A. Searching Algorithms	5
Q1. Write a program to perform Linear Search on an array.	5
Q2. Write a program to perform Linear Search on an array.	5
Q3. Write a program to perform Binary Search on a sorted array.	6
Q4. Write a program to perform Binary Search on a sorted array.	7
B. Address Calculation (1D & 2D Arrays)	9
Q5. Write a program to demonstrate address calculation in a 1D array.	9
Q6. Write a program to demonstrate address calculation in a 2D array.	10
C. Basic Array Operations (Max, Min, Frequency)	12
Q7. Write a program to find the position of the largest element in an array.	12
Q8. Write a program to find the smallest element in an array.	12
Q9. Write a program to count the number of occurrences of an element in an array.	13
D. Array Operations (Insert, Delete, Reverse)	14
Q10. Write a program to insert an element at a given position in an array.	14
Q11. Write a program to delete an element from a given position in an array.	14
Q12. Write a program to reverse an array.	15
E. Sorting Algorithms	16
Q13. Write a program to sort an array using Bubble Sort.	16
Q14. Write a program to sort an array using Selection Sort.	16
Q15. Write a program to sort an array using Insertion Sort.	17
F. Matrix Programs	19
Q16. Write a program to add two matrices.	19
Q17. Write a program to find the transpose of a matrix.	19
Q18. Write a program to find the sum of diagonal elements of a matrix.	20
G. Address Calculation Variations	22
Q19. Write a program to calculate the address of an element in a 1D array using the formula.	22
Q20. Write a program to calculate the address of an element in a 2D array using row-major order.	22
H. Stack Using Array	24
Q21. Write a program to implement Stack operations (Push, Pop, Peek, Display) using arrays.	24
Q22. Write a program to implement Stack operations (Push, Pop, Peek, Display) using arrays.	26
I. Stack Using Linked List	27
Q23. Write a program to implement Stack operations using linked list.	27
Q24. Write a program to implement Stack operations using linked list.	29
J. Postfix Expression Evaluation	30
Q25. Write a program to evaluate a postfix expression using stack.	30
Q26. Write a program to evaluate a postfix expression using stack.	32
K. Queue Using Array	33
Q27. Write a program to implement Queue operations (Enqueue, Dequeue, Display) using arrays.	33
Q28. Write a program to implement Queue operations (Enqueue, Dequeue, Display) using arrays.	35

L. Singly Linked List	36
Q29. Write a program to implement Singly Linked List (Insert, Delete, Display).	36
Q30. Write a program to implement Singly Linked List (Insert, Delete, Display).	38
M. Doubly Linked List	40
Q31. Write a program to implement Doubly Linked List (Insert, Delete, Display).	40

Quick Revision Map

Use this to jump straight to a topic instead of scrolling question by question.

Topic	Questions	Covers
Searching	Q1 – Q4	Linear Search, Binary Search
Address Calculation	Q5, Q6, Q19, Q20	1D & 2D array address formulas
Array Operations	Q7 – Q12	Max, Min, Frequency, Insert, Delete, Reverse
Sorting	Q13 – Q15	Bubble, Selection, Insertion Sort
Matrix	Q16 – Q18	Addition, Transpose, Diagonal Sum
Stack	Q21 – Q26	Array & Linked List implementation, Postfix Evaluation
Queue	Q27 – Q28	Array implementation (Enqueue / Dequeue)
Linked List	Q29 – Q31	Singly & Doubly Linked List

A. Searching Algorithms

Q1. Write a program to perform Linear Search on an array.

PROGRAM

```
#include<stdio.h>
#include<conio.h>

void main()
{
    int a[50], n, i, item, found = 0;

    clrscr();

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter elements:\n");
    for(i = 0; i < n; i++)
    {
        scanf("%d", &a[i]);
    }

    printf("Enter element to search: ");
    scanf("%d", &item);

    for(i = 0; i < n; i++)
    {
        if(a[i] == item)
        {
            printf("Element found at position %d", i + 1);
            found = 1;
            break;
        }
    }

    if(found == 0)
        printf("Element not found");

    getch();
}
```

SAMPLE RUN

```
Enter number of elements: 5
Enter elements:
10 20 30 40 50
Enter element to search: 30
Element found at position 3
```

Q2. Write a program to perform Linear Search on an array.

↳ **DUPLICATE — same as Q1. Use the program from Q1.**

This is the same question as Q1, so the same program can be written.

PROGRAM

```
#include<stdio.h>
```

```
#include<conio.h>

void main()
{
    int a[50], n, i, key;

    clrscr();

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter elements:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &a[i]);

    printf("Enter element to search: ");
    scanf("%d", &key);

    for(i = 0; i < n; i++)
    {
        if(a[i] == key)
        {
            printf("Element found at index %d", i);
            getch();
            return;
        }
    }

    printf("Element not found");

    getch();
}
```

Note: If the practical sheet genuinely contains the same question twice, you don't need to learn two different algorithms. Linear Search is one algorithm.

Q3. Write a program to perform Binary Search on a sorted array.

PROGRAM

```
#include<stdio.h>
#include<conio.h>

void main()
{
    int a[50], n, i, key;
    int low, high, mid, found = 0;

    clrscr();

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter elements in sorted order:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &a[i]);

    printf("Enter element to search: ");
    scanf("%d", &key);

    low = 0;
```

```

high = n - 1;

while(low <= high)
{
    mid = (low + high) / 2;

    if(a[mid] == key)
    {
        printf("Element found at position %d", mid + 1);
        found = 1;
        break;
    }
    else if(key < a[mid])
    {
        high = mid - 1;
    }
    else
    {
        low = mid + 1;
    }
}

if(found == 0)
    printf("Element not found");

getch();
}

```

SAMPLE RUN

```

Enter number of elements: 6
Enter elements in sorted order:
10 20 30 40 50 60
Enter element to search: 40
Element found at position 4

```

Important

Binary Search requires a sorted array.

Q4. Write a program to perform Binary Search on a sorted array.

↳ **DUPLICATE** — same as Q3. Use the program from Q3.

Again, this is a duplicate of Q3.

PROGRAM

```

#include<stdio.h>
#include<conio.h>

void main()
{
    int a[50], n, key;
    int left, right, mid;

    clrscr();

    printf("Enter number of elements: ");
    scanf("%d", &n);

```

```
printf("Enter sorted elements:\n");

{
    int i;
    for(i = 0; i < n; i++)
        scanf("%d", &a[i]);
}

printf("Enter element to search: ");
scanf("%d", &key);

left = 0;
right = n - 1;

while(left <= right)
{
    mid = (left + right) / 2;

    if(a[mid] == key)
    {
        printf("Element found at position %d", mid + 1);
        getch();
        return;
    }

    if(key < a[mid])
        right = mid - 1;
    else
        left = mid + 1;
}

printf("Element not found");

getch();
}
```

B. Address Calculation (1D & 2D Arrays)

Q5. Write a program to demonstrate address calculation in a 1D array.

For a 1D array, the address of an element is:

$$\text{LOC}(A[i]) = \text{Base} + (i - \text{LB}) \times W$$

Where:

Base = starting address of array

i = index of required element

LB = lower bound

W = size of each element in bytes

For a normal C array:

LB = 0

PROGRAM

```
#include<stdio.h>
#include<conio.h>

void main()
{
    int a[10];
    int base, i, w, address;

    clrscr();

    printf("Enter base address of array: ");
    scanf("%d", &base);

    printf("Enter size of each element in bytes: ");
    scanf("%d", &w);

    printf("Enter index of element: ");
    scanf("%d", &i);

    address = base + (i * w);

    printf("\nAddress of A[%d] = %d", i, address);

    getch();
}
```

SAMPLE RUN

```
Enter base address of array: 1000
Enter size of each element in bytes: 2
Enter index of element: 4

Address of A[4] = 1008
Because:
1000 + (4 × 2) = 1008
```

Q6. Write a program to demonstrate address calculation in a 2D array.

For a 2D array stored in row-major order:

$$\text{LOC}(A[i][j]) = \text{Base} + [(i - \text{LR}) \times N + (j - \text{LC})] \times W$$

Where:

Base = starting address

i = row number

j = column number

LR = lower row bound

LC = lower column bound

N = number of columns

W = size of each element

For normal C arrays:

LR = 0

LC = 0

So:

$$\text{LOC}(A[i][j]) = \text{Base} + [(i \times N) + j] \times W$$

PROGRAM

```
#include<stdio.h>
#include<conio.h>

void main()
{
    int a[10][10];
    int base, row, col, i, j, w, address;

    clrscr();

    printf("Enter number of rows: ");
    scanf("%d", &row);

    printf("Enter number of columns: ");
    scanf("%d", &col);

    printf("Enter base address: ");
    scanf("%d", &base);

    printf("Enter size of each element in bytes: ");
    scanf("%d", &w);

    printf("Enter row index: ");
    scanf("%d", &i);
```

```

printf("Enter column index: ");
scanf("%d", &j);

address = base + ((i * col) + j) * w;

printf("\nAddress of A[%d][%d] = %d", i, j, address);

getch();
}

```

SAMPLE RUN

Suppose:
 Rows = 3
 Columns = 4
 Base = 1000
 Element size = 2 bytes
 Required element = A[1][2]
 Then:
 $LOC(A[1][2]) = 1000 + [(1 \times 4) + 2] \times 2 = 1000 + (6 \times 2) = 1012$

C. Basic Array Operations (Max, Min, Frequency)

Q7. Write a program to find the position of the largest element in an array.

PROGRAM

```
#include<stdio.h>
#include<conio.h>

void main()
{
    int a[50], n, i, max, pos;

    clrscr();

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter elements:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &a[i]);

    max = a[0];
    pos = 0;

    for(i = 1; i < n; i++)
    {
        if(a[i] > max)
        {
            max = a[i];
            pos = i;
        }
    }

    printf("Largest element = %d", max);
    printf("\nPosition = %d", pos + 1);

    getch();
}
```

Q8. Write a program to find the smallest element in an array.

PROGRAM

```
#include<stdio.h>
#include<conio.h>

void main()
{
    int a[50], n, i, min;

    clrscr();

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter elements:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &a[i]);
}
```

```

    min = a[0];

    for(i = 1; i < n; i++)
    {
        if(a[i] < min)
            min = a[i];
    }

    printf("Smallest element = %d", min);

    getch();
}

```

Q9. Write a program to count the number of occurrences of an element in an array.

PROGRAM

```

#include<stdio.h>
#include<conio.h>

void main()
{
    int a[50], n, i, key, count = 0;

    clrscr();

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter elements:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &a[i]);

    printf("Enter element to count: ");
    scanf("%d", &key);

    for(i = 0; i < n; i++)
    {
        if(a[i] == key)
            count++;
    }

    printf("Element occurs %d times", count);

    getch();
}

```

D. Array Operations (Insert, Delete, Reverse)

Q10. Write a program to insert an element at a given position in an array.

PROGRAM

```
#include<stdio.h>
#include<conio.h>

void main()
{
    int a[50], n, i, pos, value;

    clrscr();

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter elements:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &a[i]);

    printf("Enter position: ");
    scanf("%d", &pos);

    printf("Enter value: ");
    scanf("%d", &value);

    for(i = n; i >= pos; i--)
        a[i] = a[i - 1];

    a[pos - 1] = value;
    n++;

    printf("Array after insertion:\n");

    for(i = 0; i < n; i++)
        printf("%d ", a[i]);

    getch();
}
```

Q11. Write a program to delete an element from a given position in an array.

PROGRAM

```
#include<stdio.h>
#include<conio.h>

void main()
{
    int a[50], n, i, pos;

    clrscr();

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter elements:\n");
```

```

    for(i = 0; i < n; i++)
        scanf("%d", &a[i]);

    printf("Enter position to delete: ");
    scanf("%d", &pos);

    for(i = pos - 1; i < n - 1; i++)
        a[i] = a[i + 1];

    n--;

    printf("Array after deletion:\n");

    for(i = 0; i < n; i++)
        printf("%d ", a[i]);

    getch();
}

```

Q12. Write a program to reverse an array.

PROGRAM

```

#include<stdio.h>
#include<conio.h>

void main()
{
    int a[50], n, i, temp;

    clrscr();

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter elements:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &a[i]);

    for(i = 0; i < n / 2; i++)
    {
        temp = a[i];
        a[i] = a[n - 1 - i];
        a[n - 1 - i] = temp;
    }

    printf("Reversed array:\n");

    for(i = 0; i < n; i++)
        printf("%d ", a[i]);

    getch();
}

```

E. Sorting Algorithms

Q13. Write a program to sort an array using Bubble Sort.

PROGRAM

```
#include<stdio.h>
#include<conio.h>

void main()
{
    int a[50], n, i, j, temp;

    clrscr();

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter elements:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &a[i]);

    for(i = 0; i < n - 1; i++)
    {
        for(j = 0; j < n - 1 - i; j++)
        {
            if(a[j] > a[j + 1])
            {
                temp = a[j];
                a[j] = a[j + 1];
                a[j + 1] = temp;
            }
        }
    }

    printf("Sorted array:\n");

    for(i = 0; i < n; i++)
        printf("%d ", a[i]);

    getch();
}
```

Q14. Write a program to sort an array using Selection Sort.

PROGRAM

```
#include<stdio.h>
#include<conio.h>

void main()
{
    int a[50], n, i, j, min, temp;

    clrscr();

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter elements:\n");
```

```

for(i = 0; i < n; i++)
    scanf("%d", &a[i]);

for(i = 0; i < n - 1; i++)
{
    min = i;

    for(j = i + 1; j < n; j++)
    {
        if(a[j] < a[min])
            min = j;
    }

    temp = a[i];
    a[i] = a[min];
    a[min] = temp;
}

printf("Sorted array:\n");

for(i = 0; i < n; i++)
    printf("%d ", a[i]);

getch();
}

```

Q15. Write a program to sort an array using Insertion Sort.

PROGRAM

```

#include<stdio.h>
#include<conio.h>

void main()
{
    int a[50], n, i, j, key;

    clrscr();

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter elements:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &a[i]);

    for(i = 1; i < n; i++)
    {
        key = a[i];
        j = i - 1;

        while(j >= 0 && a[j] > key)
        {
            a[j + 1] = a[j];
            j--;
        }

        a[j + 1] = key;
    }
}

```

```
printf("Sorted array:\n");  
  
for(i = 0; i < n; i++)  
    printf("%d ", a[i]);  
  
getch();  
}
```

F. Matrix Programs

Q16. Write a program to add two matrices.

PROGRAM

```
#include<stdio.h>
#include<conio.h>

void main()
{
    int a[10][10], b[10][10], c[10][10];
    int r, col, i, j;

    clrscr();

    printf("Enter rows and columns: ");
    scanf("%d%d", &r, &col);

    printf("Enter first matrix:\n");

    for(i = 0; i < r; i++)
        for(j = 0; j < col; j++)
            scanf("%d", &a[i][j]);

    printf("Enter second matrix:\n");

    for(i = 0; i < r; i++)
        for(j = 0; j < col; j++)
            scanf("%d", &b[i][j]);

    for(i = 0; i < r; i++)
        for(j = 0; j < col; j++)
            c[i][j] = a[i][j] + b[i][j];

    printf("Result:\n");

    for(i = 0; i < r; i++)
    {
        for(j = 0; j < col; j++)
            printf("%d ", c[i][j]);

        printf("\n");
    }

    getch();
}
```

Q17. Write a program to find the transpose of a matrix.

PROGRAM

```
#include<stdio.h>
#include<conio.h>

void main()
{
    int a[10][10], t[10][10];
    int r, c, i, j;
```

```

clrscr();

printf("Enter rows and columns: ");
scanf("%d%d", &r, &c);

printf("Enter matrix:\n");

for(i = 0; i < r; i++)
    for(j = 0; j < c; j++)
        scanf("%d", &a[i][j]);

for(i = 0; i < r; i++)
    for(j = 0; j < c; j++)
        t[j][i] = a[i][j];

printf("Transpose:\n");

for(i = 0; i < c; i++)
{
    for(j = 0; j < r; j++)
        printf("%d ", t[i][j]);

    printf("\n");
}

getch();
}

```

Q18. Write a program to find the sum of diagonal elements of a matrix.

PROGRAM

```

#include<stdio.h>
#include<conio.h>

void main()
{
    int a[10][10], n, i, j, sum = 0;

    clrscr();

    printf("Enter order of matrix: ");
    scanf("%d", &n);

    printf("Enter matrix:\n");

    for(i = 0; i < n; i++)
        for(j = 0; j < n; j++)
            scanf("%d", &a[i][j]);

    for(i = 0; i < n; i++)
        sum = sum + a[i][i];

    printf("Sum of main diagonal = %d", sum);

    getch();
}

```

G. Address Calculation Variations

Q19. Write a program to calculate the address of an element in a 1D array using the formula.

PROGRAM

```
#include<stdio.h>
#include<conio.h>

void main()
{
    int base, index, lower, size, address;

    clrscr();

    printf("Enter base address: ");
    scanf("%d", &base);

    printf("Enter lower bound: ");
    scanf("%d", &lower);

    printf("Enter index: ");
    scanf("%d", &index);

    printf("Enter size of each element: ");
    scanf("%d", &size);

    address = base + (index - lower) * size;

    printf("Address = %d", address);

    getch();
}
```

Q20. Write a program to calculate the address of an element in a 2D array using row-major order.

PROGRAM

```
#include<stdio.h>
#include<conio.h>

void main()
{
    int base, i, j, lr, lc, n, w, address;

    clrscr();

    printf("Enter base address: ");
    scanf("%d", &base);

    printf("Enter row index: ");
    scanf("%d", &i);

    printf("Enter column index: ");
    scanf("%d", &j);

    printf("Enter lower row bound: ");
    scanf("%d", &lr);
```

```
printf("Enter lower column bound: ");
scanf("%d", &lc);

printf("Enter number of columns: ");
scanf("%d", &n);

printf("Enter size of each element: ");
scanf("%d", &w);

address = base + (((i - lr) * n) + (j - lc)) * w;

printf("Address = %d", address);

getch();
}
```

H. Stack Using Array

Q21. Write a program to implement Stack operations (Push, Pop, Peek, Display) using arrays.

PROGRAM

```
#include<stdio.h>
#include<conio.h>

#define MAX 50

int stack[MAX];
int top = -1;

void push()
{
    int value;

    if(top == MAX - 1)
    {
        printf("\nStack Overflow");
    }
    else
    {
        printf("\nEnter value: ");
        scanf("%d", &value);

        top++;
        stack[top] = value;

        printf("Element pushed successfully");
    }
}

void pop()
{
    if(top == -1)
    {
        printf("\nStack Underflow");
    }
    else
    {
        printf("\nDeleted element = %d", stack[top]);
        top--;
    }
}

void peek()
{
    if(top == -1)
        printf("\nStack is empty");
    else
        printf("\nTop element = %d", stack[top]);
}

void display()
{
    int i;

    if(top == -1)
    {
```

```

        printf("\nStack is empty");
    }
    else
    {
        printf("\nStack elements:\n");

        for(i = top; i >= 0; i--)
            printf("%d\n", stack[i]);
    }
}

void main()
{
    int choice;

    clrscr();

    do
    {
        printf("\n\n--- STACK MENU ---");
        printf("\n1. Push");
        printf("\n2. Pop");
        printf("\n3. Peek");
        printf("\n4. Display");
        printf("\n5. Exit");

        printf("\nEnter your choice: ");
        scanf("%d", &choice);

        switch(choice)
        {
            case 1:
                push();
                break;

            case 2:
                pop();
                break;

            case 3:
                peek();
                break;

            case 4:
                display();
                break;

            case 5:
                printf("\nExiting...");
                break;

            default:
                printf("\nInvalid choice");
        }

    } while(choice != 5);

    getch();
}

```

Important concepts

Stack follows LIFO:

Last In, First Out

Example:

Push 10

Push 20

Push 30

Top

↓

30

20

10

Pop() removes 30 first.

Stack conditions

Overflow:

$top == MAX - 1$

Underflow:

$top == -1$

Q22. Write a program to implement Stack operations (Push, Pop, Peek, Display) using arrays.

↳ **DUPLICATE** — same as Q21. Use the program from Q21.

This is the same question as Q21.

You can write the same program.

Don't learn a second program for Q22.

I. Stack Using Linked List

Q23. Write a program to implement Stack operations using linked list.

PROGRAM

```
#include<stdio.h>
#include<conio.h>
#include<stdlib.h>

struct node
{
    int data;
    struct node *next;
};

struct node *top = NULL;

void push()
{
    struct node *newnode;
    int value;

    newnode = (struct node *)malloc(sizeof(struct node));

    printf("\nEnter value: ");
    scanf("%d", &value);

    newnode->data = value;
    newnode->next = top;
    top = newnode;

    printf("Element pushed successfully");
}

void pop()
{
    struct node *temp;

    if(top == NULL)
    {
        printf("\nStack Underflow");
    }
    else
    {
        temp = top;

        printf("\nDeleted element = %d", top->data);

        top = top->next;

        free(temp);
    }
}

void peek()
{
    if(top == NULL)
        printf("\nStack is empty");
    else
```

```

        printf("\nTop element = %d", top->data);
    }

void display()
{
    struct node *temp;

    if(top == NULL)
    {
        printf("\nStack is empty");
    }
    else
    {
        temp = top;

        printf("\nStack elements:\n");

        while(temp != NULL)
        {
            printf("%d\n", temp->data);
            temp = temp->next;
        }
    }
}

void main()
{
    int choice;

    clrscr();

    do
    {
        printf("\n\n--- STACK USING LINKED LIST ---");
        printf("\n1. Push");
        printf("\n2. Pop");
        printf("\n3. Peek");
        printf("\n4. Display");
        printf("\n5. Exit");

        printf("\nEnter your choice: ");
        scanf("%d", &choice);

        switch(choice)
        {
            case 1:
                push();
                break;

            case 2:
                pop();
                break;

            case 3:
                peek();
                break;

            case 4:
                display();
                break;
        }
    }
}

```

```
        case 5:
            break;

        default:
            printf("\nInvalid choice");
    }

    while(choice != 5);

    getch();
}
```

Q24. Write a program to implement Stack operations using linked list.

↳ **DUPLICATE** — same as Q23. Use the program from Q23.

This is a duplicate of Q23.

Use the same program.

J. Postfix Expression Evaluation

Q25. Write a program to evaluate a postfix expression using stack.

For simplicity, this program evaluates a postfix expression containing single-digit operands.

Example:

23*54*+

Meaning:

$(2 \times 3) + (5 \times 4)$

$= 6 + 20$

$= 26$

PROGRAM

```
#include<stdio.h>
#include<conio.h>
#include<ctype.h>

int stack[50];
int top = -1;

void push(int value)
{
    top++;
    stack[top] = value;
}

int pop()
{
    int value;

    value = stack[top];
    top--;

    return value;
}

void main()
{
    char postfix[50];
    int i;
    int a, b, result;

    clrscr();

    printf("Enter postfix expression: ");
    scanf("%s", postfix);

    for(i = 0; postfix[i] != '\0'; i++)
    {
        if(isdigit(postfix[i]))
        {
            push(postfix[i] - '0');
        }
    }
}
```

```

else
{
    b = pop();
    a = pop();

    switch(postfix[i])
    {
        case '+':
            result = a + b;
            break;

        case '-':
            result = a - b;
            break;

        case '*':
            result = a * b;
            break;

        case '/':
            result = a / b;
            break;

        case '%':
            result = a % b;
            break;
    }

    push(result);
}
}

printf("\nResult = %d", pop());

getch();
}

```

SAMPLE RUN

Enter postfix expression: 23*54*+
 Result = 26

Very important for viva

For postfix:

Operand → PUSH

Operator → POP two operands → perform operation → PUSH result

And remember the order:

b = pop();

a = pop();

a - b

a / b

Not:

b - a

b / a

Q26. Write a program to evaluate a postfix expression using stack.

↳ **DUPLICATE — same as Q25. Use the program from Q25.**

This is a duplicate of Q25.

Use the same program.

K. Queue Using Array

Q27. Write a program to implement Queue operations (Enqueue, Dequeue, Display) using arrays.

PROGRAM

```
#include<stdio.h>
#include<conio.h>

#define MAX 50

int queue[MAX];
int front = -1;
int rear = -1;

void enqueue()
{
    int value;

    if(rear == MAX - 1)
    {
        printf("\nQueue Overflow");
    }
    else
    {
        printf("\nEnter value: ");
        scanf("%d", &value);

        if(front == -1)
            front = 0;

        rear++;
        queue[rear] = value;

        printf("Element inserted successfully");
    }
}

void dequeue()
{
    if(front == -1 || front > rear)
    {
        printf("\nQueue Underflow");
    }
    else
    {
        printf("\nDeleted element = %d", queue[front]);
        front++;

        if(front > rear)
        {
            front = -1;
            rear = -1;
        }
    }
}

void display()
{
    int i;
```

```

    if(front == -1)
    {
        printf("\nQueue is empty");
    }
    else
    {
        printf("\nQueue elements:\n");

        for(i = front; i <= rear; i++)
            printf("%d ", queue[i]);
    }
}

void main()
{
    int choice;

    clrscr();

    do
    {
        printf("\n\n--- QUEUE MENU ---");
        printf("\n1. Enqueue");
        printf("\n2. Dequeue");
        printf("\n3. Display");
        printf("\n4. Exit");

        printf("\nEnter your choice: ");
        scanf("%d", &choice);

        switch(choice)
        {
            case 1:
                enqueue();
                break;

            case 2:
                dequeue();
                break;

            case 3:
                display();
                break;

            case 4:
                break;

            default:
                printf("\nInvalid choice");
        }

    } while(choice != 4);

    getch();
}

```

Important concept*Queue follows FIFO:**First In, First Out*

Example:

Enqueue: 10 20 30

Front Rear

↓

↓

10 20 30

Dequeue() removes 10 first.

Q28. Write a program to implement Queue operations (Enqueue, Dequeue, Display) using arrays.

↳ **DUPLICATE** — same as Q27. Use the program from Q27.

This is a duplicate of Q27.

Use the same program.

L. Singly Linked List

Q29. Write a program to implement Singly Linked List (Insert, Delete, Display).

PROGRAM

```
#include<stdio.h>
#include<conio.h>
#include<stdlib.h>

struct node
{
    int data;
    struct node *next;
};

struct node *head = NULL;

void insert()
{
    struct node *newnode, *temp;
    int value;

    newnode = (struct node *)malloc(sizeof(struct node));

    printf("\nEnter value: ");
    scanf("%d", &value);

    newnode->data = value;
    newnode->next = NULL;

    if(head == NULL)
    {
        head = newnode;
    }
    else
    {
        temp = head;

        while(temp->next != NULL)
            temp = temp->next;

        temp->next = newnode;
    }

    printf("Node inserted successfully");
}

void deleteNode()
{
    struct node *temp, *prev;
    int value;

    if(head == NULL)
    {
        printf("\nList is empty");
        return;
    }

    printf("\nEnter value to delete: ");
```

```

scanf("%d", &value);

if(head->data == value)
{
    temp = head;
    head = head->next;

    free(temp);

    printf("Node deleted successfully");
    return;
}

temp = head;

while(temp != NULL && temp->data != value)
{
    prev = temp;
    temp = temp->next;
}

if(temp == NULL)
{
    printf("\nElement not found");
}
else
{
    prev->next = temp->next;
    free(temp);

    printf("Node deleted successfully");
}
}

void display()
{
    struct node *temp;

    if(head == NULL)
    {
        printf("\nList is empty");
    }
    else
    {
        temp = head;

        printf("\nLinked List:\n");

        while(temp != NULL)
        {
            printf("%d -> ", temp->data);
            temp = temp->next;
        }

        printf("NULL");
    }
}

void main()
{
    int choice;

```

```

clrscr();

do
{
    printf("\n\n--- SINGLY LINKED LIST ---");
    printf("\n1. Insert");
    printf("\n2. Delete");
    printf("\n3. Display");
    printf("\n4. Exit");

    printf("\nEnter your choice: ");
    scanf("%d", &choice);

    switch(choice)
    {
        case 1:
            insert();
            break;

        case 2:
            deleteNode();
            break;

        case 3:
            display();
            break;

        case 4:
            break;

        default:
            printf("\nInvalid choice");
    }

} while(choice != 4);

getch();
}

```

SAMPLE RUN

```
10 -> 20 -> 30 -> NULL
```

Q30. Write a program to implement Singly Linked List (Insert, Delete, Display).

↳ **DUPLICATE — same as Q29. Use the program from Q29.**

This is a duplicate of Q29.

Use the same program.

M. Doubly Linked List

Q31. Write a program to implement Doubly Linked List (Insert, Delete, Display).

PROGRAM

```
#include<stdio.h>
#include<conio.h>
#include<stdlib.h>

struct node
{
    int data;
    struct node *prev;
    struct node *next;
};

struct node *head = NULL;

void insert()
{
    struct node *newnode, *temp;
    int value;

    newnode = (struct node *)malloc(sizeof(struct node));

    printf("\nEnter value: ");
    scanf("%d", &value);

    newnode->data = value;
    newnode->prev = NULL;
    newnode->next = NULL;

    if(head == NULL)
    {
        head = newnode;
    }
    else
    {
        temp = head;

        while(temp->next != NULL)
            temp = temp->next;

        temp->next = newnode;
        newnode->prev = temp;
    }

    printf("Node inserted successfully");
}

void deleteNode()
{
    struct node *temp;
    int value;

    if(head == NULL)
    {
        printf("\nList is empty");
        return;
    }
}
```

```

printf("\nEnter value to delete: ");
scanf("%d", &value);

temp = head;

while(temp != NULL && temp->data != value)
    temp = temp->next;

if(temp == NULL)
{
    printf("\nElement not found");
    return;
}

if(temp->prev != NULL)
    temp->prev->next = temp->next;
else
    head = temp->next;

if(temp->next != NULL)
    temp->next->prev = temp->prev;

free(temp);

printf("Node deleted successfully");
}

void display()
{
    struct node *temp;

    if(head == NULL)
    {
        printf("\nList is empty");
    }
    else
    {
        temp = head;

        printf("\nDoubly Linked List:\n");

        while(temp != NULL)
        {
            printf("%d <-> ", temp->data);
            temp = temp->next;
        }

        printf("NULL");
    }
}

void main()
{
    int choice;

    clrscr();

    do
    {

```

```

printf("\n\n--- DOUBLY LINKED LIST ---");
printf("\n1. Insert");
printf("\n2. Delete");
printf("\n3. Display");
printf("\n4. Exit");

printf("\nEnter your choice: ");
scanf("%d", &choice);

switch(choice)
{
    case 1:
        insert();
        break;

    case 2:
        deleteNode();
        break;

    case 3:
        display();
        break;

    case 4:
        break;

    default:
        printf("\nInvalid choice");
}

} while(choice != 4);

getch();
}

```

SAMPLE RUN

```
10 <-> 20 <-> 30 <-> NULL
```

The important difference from a singly linked list is that every node has two links:

```
prev | data | next
```